Core Design Patterns with Non-Software Examples

Abstract

Software design patterns have roots in the architectural patterns of Christopher Alexander, and in the object movement. According to Alexander, patterns repeat themselves, since they are a generic solution to a given system of forces. The object movement looks to the real world for insights into modeling software relationships. With these dual roots, it seems reasonable that software design patterns should be repeated in real world objects. This paper presents a real world, non software instance of each design pattern from the book, Design Patterns - Elements of Reusable Object-Oriented Software [13]. The paper also discusses the implications of non-software examples on the communicative power of a pattern language, and on design pattern training.

0 Introduction

Within the software industry, a growing community of patterns proponents exists. The roots of the patterns movement are found in the writings of architect Christopher Alexander, who describes a pattern as a generic solution to a given system of forces in the world [1]. Alexander's patterns can be observed in everyday structures. Each pattern in A Pattern Language [2] includes a picture of an archetypal example of the pattern.

Since objects were the predominate world view at the time that patterns were embraced by the software world, patterns also have roots in the object movement [9]. Unfortunately examples of software design patterns are not as abundant as Alexandrian patterns, since they represent elegant designs, rather than the designs that people generate initially [13]. Access to elegant designs is often limited due to the proprietary nature of much of the software being developed today.

According to Alexander, real world patterns always repeat themselves, because under a given set of circumstances, there are always certain fields of relationships which are most nearly well adapted to the forces which exist [1]. In software, real world problems are either modeled entirely, or real world objects are transformed into hardware and software to produce real world results [5]. Since software design patterns have roots in both Alexandrian patterns, and in the object movement, it seems logical that software design patterns can be found in real world objects. This is not to say that software design patterns are necessarily models of the real world objects, but the relationships between objects that have been adapted to deal with certain forces can be observed both in the "real world" and in software objects. To test this hypothesis, a real world example was sought for each of the 23 Gang of Four Patterns [13].

1 What are design patterns?

1. Recurring solutions to design problems we see over
2. Constitutes a set of rules to accomplish certain task in software development realm
3. Convenient ways of reusing object oriented code between projects and between programmers.
4. These describe how different objects should interact with each other without getting entangled with others data model and methods.
5. Started from early 1980, formally recognized in 1990(Helm) and 1992 (Enrich Gamma)
6. Suggests to program to an interface not to an implementation
7. Recommends composition over inheritance

2 Types of Core Design Patterns

1. Creational – These help us to create the objects instead of directly instantiating the instances and give the flexibility to create the objects depending on the case.
2. Structural – Help us to compose a number of objects into a large structure.
3. Behavioral – Help us to define the communication between objects and flow of control in the system.

Total number of generally used core patterns: 23. This document may not cover all the 23 patterns but will cover most of the generally used patterns.

3 Creational Patterns

The Gang of Four has documented five creational patterns. Examples of these creational patterns can be found in manufacturing, fast food, biology and political institutions.

3.1 Factory Pattern

It is responsible for creating the objects.

As name depicts, it is a factory for creating any specific kind of objects. It abstracts the user from the details of creation mechanism, and also abstracts the actual implementation class of instantiated object. The only requirement is that the created object should be of some already defined and accepted type. The actual implementation may differ based on current scenario.

OR

The Factory Method defines an interface for creating objects, but lets subclasses decide which classes to instantiate. Injection molding presses demonstrate this pattern. Manufacturers of plastic toys process plastic molding powder, and inject the plastic into molds of the desired shapes [15]. The class of toy (car, action figure, etc.) is determined by the mold.

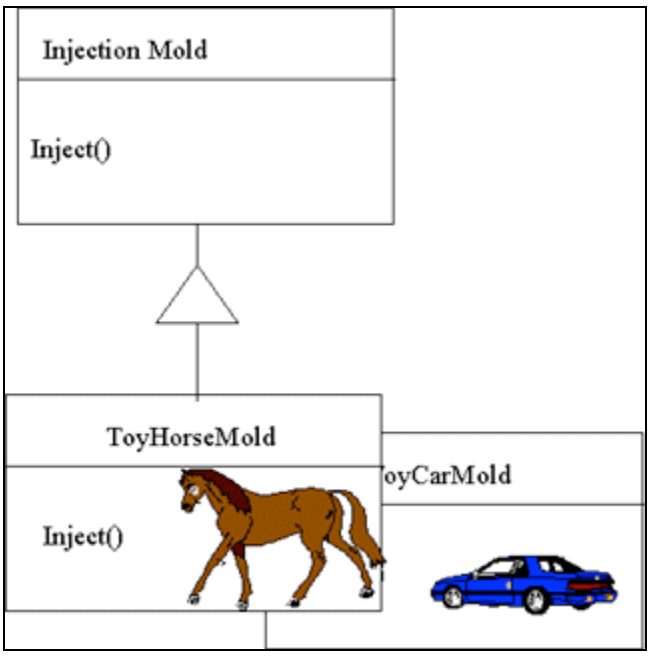

Figure 1: Object Diagram for Factory Method using Injection Mold Example

3.1.1 Example

1. Maruti Car Factory Object: We can ask this object to return us a Maruti car based on any runtime parameter which defines its model like Wagon R, or Esteem. It is the responsibility of this factory object to return us required car implementation based on specific parameters.
2. Row Renderer Factory Object: We can ask it to return the renderer like structured row renderer, or a designable row renderer.

3.1.2 When to Use

1. A class can't anticipate which class of object it must create.
2. We want to localized the knowledge of object creation into some class
3. We want to make the system configurable and extensible and so want to program to interface not to implementation.

3.1.3 Usage

1. We are free from object creation and choosing the right implementing class.
2. The mechanism for choosing the right implementing class can be made very configurable by using some configuration files (INI or XML).
3. Programming to interface rather than to concrete class so enabling extensibility of the system.

3.2 Abstract Factory Pattern

It provides an interface to create and return one of the several families of related objects. It can also be referred as factory of factory objects. **It is one level of higher abstraction than Factory Pattern. It is a factory object which can return one of the several factory objects.**

OR

The purpose of the Abstract Factory is to provide an interface for creating families of related objects, without specifying concrete classes. This pattern is found in the sheet metal stamping equipment used in the manufacture of Japanese automobiles. The stamping equipment is an Abstract Factory which creates auto body parts. The same machinery is used to stamp right hand doors, left hand doors, right front fenders, left front fenders, hoods etc. for different models of cars. Through the use of rollers to change the stamping dies, the concrete classes produced by the machinery can be changed within three minutes.

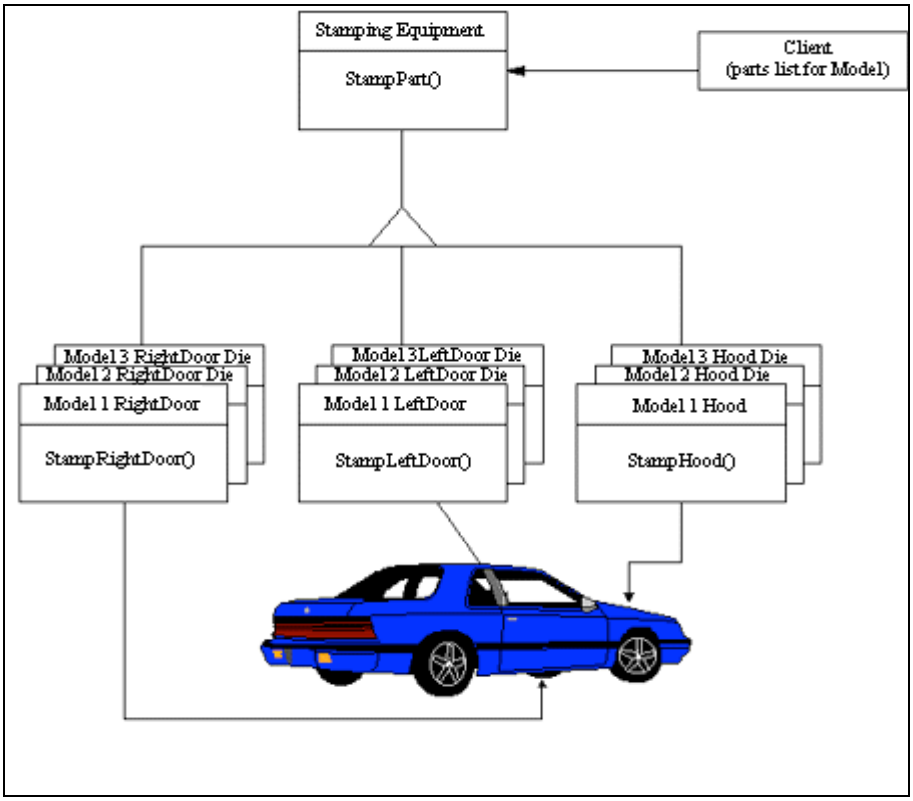

Figure 2: Stamping Example of the Abstract Factory

3.2.1 Example

1. Car Factory Object: We can ask it for returning as a specific type of car factory object like Maruti Car Factory, Hyundai Car Factory or GM Car Factory. Further the return specific car factories can be used to return cars of different model for respective brands.
2. Renderer Factory: We can ask it to return factory object for a specific purpose renderer like Row Renderer, Column Renderer, or Header Renderer Factory. Further these factories can be used to return specific type of renderer of different implementation as defined with Factory Pattern.

3.2.2 When to use

1. About same as mentioned with factory pattern.

3.2.3 Usage

1. It isolates the concrete classes that are generated; due to this it is very easy to change the implementation.
2. Rest almost same as mentioned with factory pattern.

3.3 Singleton Pattern

It is a class of which there can not be more than one instance. It provides a single global point of access to that instance.

OR

The Singleton pattern ensures that a class has only one instance, and provides a global point of access to that instance. The Singleton pattern is named after the singleton set, which is defined to be a set containing one element. **The office of the President of the United States is a Singleton.** The United States Constitution specifies the means by which a president is elected, limits the term of office, and defines the order of succession. As a result, there can be at most one active president at any given time. Regardless of the personal identity of the active president, the title, "The President of the United States" is a global point of access that identifies the person in the office.

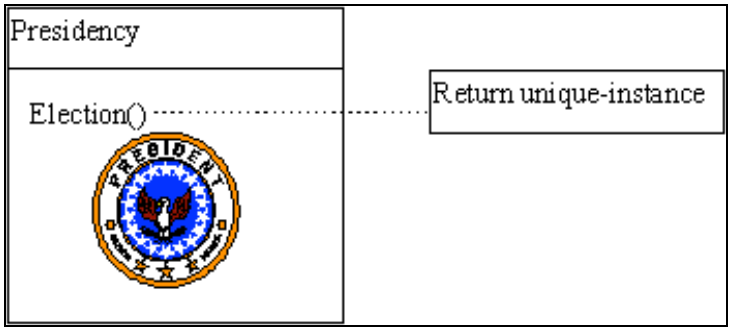

Figure 3: Object Diagram for Singleton using Presidency Example

3.3.1 Example

1. Any Utility Class: Generally utility classes do not have states with these and so a single instance of that class can serve the whole system.
2. Memory Manager: An object which is responsible to manage the memory in system. Obviously only there should be only one instance of this object so that it can update the objects with the current memory availability and other objects can update its state for expected memory requirement.

3.3.2 When to Use

1. When only one instance can solve the purpose like utility classes.
2. When only one instance is must to preserve and synchronize the entire system states.

3.3.3 Usage

1. Helps to ensure only one object in memory.
2. Helps to share the states across the system.

3.3.4 Drawbacks

1. Difficult to extend any singleton class as it will work only if super class has not been instantiated by now.

3.4 Builder Pattern:

It separates the construction of complex object from its representation, so that number of different representations can be build based on the requirement.

It is similar to factory pattern, but it does not return simple descendant of base class. The factory is concerned with what is made and the builder with how it is made. Factory simply returns an instance of related class of base type, but builder returns a composition of various classes depending on the specified state.

OR

The Builder pattern separates the construction of a complex object from its representation, so that the same construction process can create different representation. This pattern is used by fast food restaurants to construct children's meals. Children's meals typically consist of a main item, a side item, a drink, and a toy (e.g., a hamburger, fries, coke, and toy car). Note that there can be variation in the contents of the children's meal, but the construction process is the same. Whether a customer orders a hamburger, cheeseburger, or chicken, the process is the same. The employee at the counter directs the crew to assemble a main item, side item, and toy. These items are then placed in a bag. The drink is placed in a cup and remains outside of the bag. This same process is used at competing restaurants.

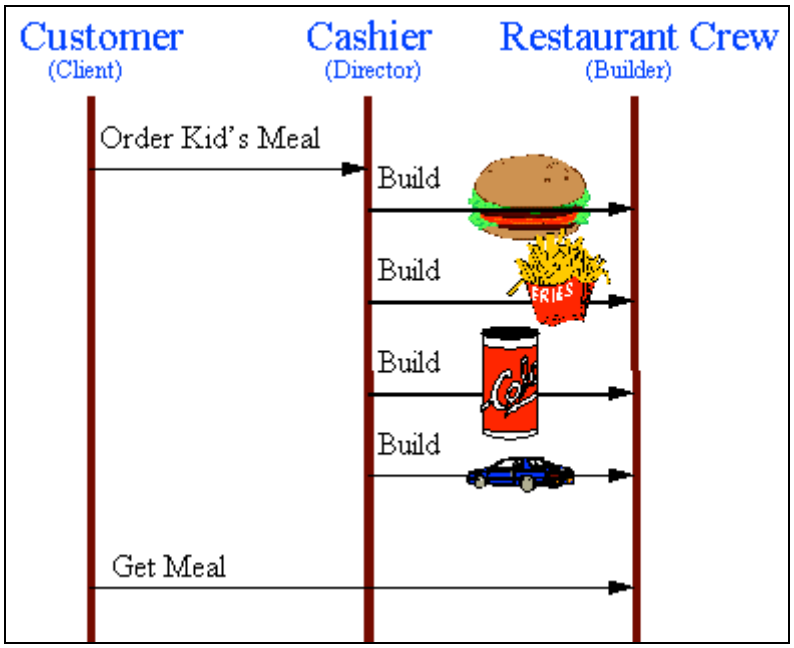

Figure 4: Object Interaction Diagram for the Builder using Kid's Meal Example

3.4.5 Example

1. Editor Builder: Suppose you are creating a UI for workflow where a number of different types of nodes can exist. We need different editors to edit these nodes, which can be very complex in structure. Here we can use builder which can build the desired editor. Even editors can vary in structure based on current state of system.

3.4.6 When to Use

1. When we have a complex UI element to build whose structure and composition may vary depending on the system state.
2. When we want to isolate and abstract the complex UI building mechanism from rest of the system.

3.4.7 Usage

1. A Builder helps you vary the internal presentation of the product it builds.
2. It hides the detail of how the product is assembled.
3. Builder build the final product step by step based on data given, so we have more control over the final product.

3.5 Prototype Pattern

It suggests, instead of creating a new instance of any object, make a copy of existing object and modify its properties as required.

Generally cloning is used to create the copy of an existing object.

OR

The Prototype pattern specifies the kind of objects to create using a prototypical instance. Prototypes of new products are often built prior to full production, but in this example, the prototype is passive, and does not participate in copying itself. The mitotic division of a cell, resulting in two identical cells, is an example of a prototype that plays an active

role in copying itself and thus, demonstrates the Prototype pattern. When a cell splits, two cells of identical genotype result. In other words, the cell clones itself.

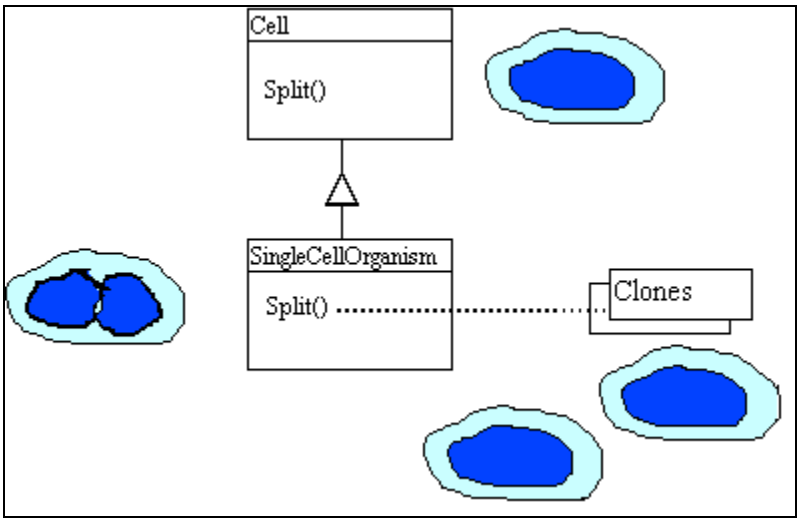

Figure 5: Object Diagram for Prototype using Cell Division Example

3.5.1 Example

1. Drag 'n Drop: While implementing drag and drop functionality in a UI, we need to show the shadow of items under dragging operation so that user can decide the place to drop the dragged item. Here instead of creating a new instance of item and setting the required properties, best way is to create a prototype of existing object and modify the required properties.
2. Filtered data set using same result set – We retrieved a large data as result set from database. Now we want to show different views on UI based on specified filter criteria. One way is to fetch the same data for every filter criteria, another way is to make the prototype of result set and apply filter to it. It will be less costly than repetitive database access.

3.5.2 When to Use

1. When creating a new instance is much costly than creating the copy of an existing object like in case of result set.
2. When we need an exact copy of any object.

3.5.3 Usage

1. Avoid the cost of recreating an object

3.5.4 Drawbacks

1. Difficulty in adding clone or deep clone method in already existing classes.
2. If we are creating a prototype registry for easy access, a new instance of every object is required, which can be a performance overhead.
3. To copy the data of existing object, we should have sufficient access to the data or method of the class.

4 Structural Patterns

The Gang of Four has documented seven structural patterns. Examples of these patterns can be found in hand tools, residential wiring, mathematics, holiday tradition, catalog retail, and banking.

4.1 Adapter Pattern

Adapter is used to adapt the behavior/functionalities of an existing class but with a different interface.

It can be implemented by two ways, by inheritance (also called class adapter) or by composition (also called object adapter). Either we can extend the existing class and use its functionalities in new interface method, or we can use a reference of existing class in new object for using existing functionalities.

OR

The Adapter pattern allows otherwise incompatible classes to work together by converting the interface of one class into an interface expected by the clients. Socket wrenches provide an example of the Adapter. A socket attaches to a ratchet, provided that the size of the drive is the same. Typical drive sizes in the United States are 1/2" and 1/4". Obviously a 1/2" drive ratchet will not fit into a 1/4" drive socket unless an adapter is used. A 1/2" to 1/4" adapter has a 1/2" female connection to fit on the 1/2" drive ratchet, and a 1/4" male connection to fit in the 1/4" drive socket.

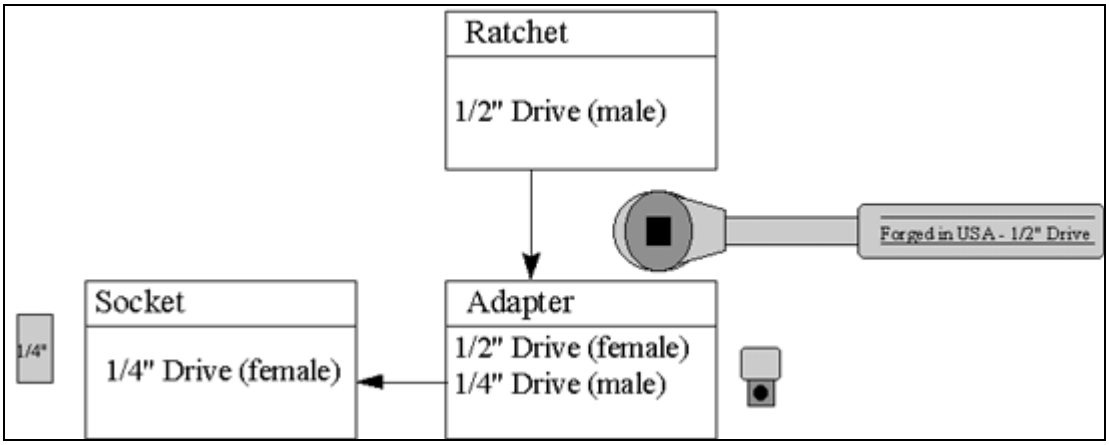

Figure 6: Object Diagram for Adapter using Socket Adapter Example

4.1.1 Example

1. We are making data bound UI components, which are implementing some of our proprietary interface to support the data loading/unloading. But UI components are already defined with Swing library so we want to use these and just want to bind them with data sources. Here we can extend our new components from existing Swing component and implement our data binding interface to it.
2. Use of Window Adapter while intercepting the window closing event.

4.1.2 When to use

1. Sometime in our system, we use any class/interface whose some/most of the functionalities are already defined with any other existing class. But due to system protocol, we have to use only new class probably with some new functionality. Here we can apply adapter pattern to adapt the behavior of existing class but with new interface.

4.1.3 Usage

1. Can use the functionalities of existing classes but with new interface without re-implementing it.
2. Object adapter even help to adapt the sub-class implementations of a class by simply referring it.

4.1.4 Drawbacks

1. Class adapter won't work when we want to adapt a class and all of its sub classes because we need to derive the class from adapted classes while creating the new class.

4.2 Bridge Pattern

It is used to separate the interface from its implementation and enables us to change either interface or implementation easily. Generally it is used to keep the client interface constant and even if we change the implementation.

It seems quite similar to adapter pattern, where we can adapt the functionalities of any other class and expose this with a separate interface. But the difference is in purpose, Adapter is implemented to use the existing implementation with a new interface but Bridge is used to separate the implementation from interface to user so that user won't be affected by any change in implementation.

OR

The Bridge pattern decouples an abstraction from its implementation, so that the two can vary independently. A household switch controlling lights, ceiling fans, etc. is an example of the Bridge. The purpose of the switch is to turn a device on or off. The actual switch can be implemented as a pull chain, a simple two position switch, or a variety of dimmer switches.

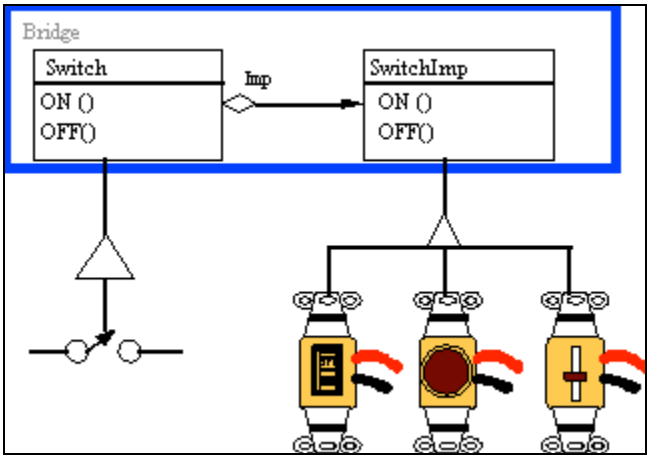

Figure 7: Object Diagram for Bridge using Electrical Switch Example

4.2.1 Example

1. We want to present some data to user. Based on the user profile, the data can be either in simple list form having names only or in a table having complete details. Here we can implement the logic to fetch the data and arranging it in required format in a separate class which can be accessed from user interface which could be a list, table or tree.

4.2.2 When to use

1. When implementation of functionality can be used by multiple interfaces and we don't want to duplicate the code for each interface.
2. When we want to keep the user interface intact from any change in implementation.

4.2.3 Usage

1. Avoid change in interface on any change in implementation

2. Help to avoid the duplicate code for different interfaces having similar kind of back end requirements.
3. It can prevent us to recompile the user interface on any change in implementation, but only bridge and implementation can be recompiled.

4.3 Composite Pattern

It is a collection of objects, anyone of which can be either a composite or just a primitive object. It enables us to represent a single object or a collection of objects using the same interface.

OR

The Composite composes objects into tree structures, and lets clients treat individual objects and compositions uniformly. Although the example is abstract, arithmetic expressions are Composites. An arithmetic expression consists of an operand, an operator (+ - * /), and another operand. The operand can be a number, or another arithmetic expression. Thus, 2 + 3 and (2 + 3) + (4 * 6) are both valid expressions.

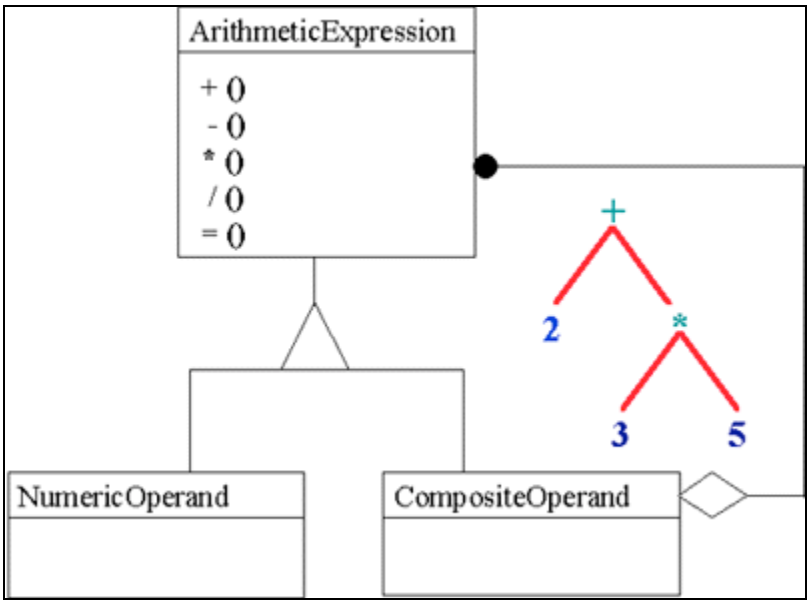

Figure 8: Object Diagram for Composite using Arithmetic Expression Example

4.3.1 Example

1. If we are designing a runtime system structure for java program, we can implement it in a tree form where every node can be a executable piece of statement and it may have other nodes in it (which should be executed before it) or may be a leaf node, which just need to execute itself and return the result to the parent (or makes the changes in shared state cache).
2. If we are designing a system to display the employee structuring of an organization. It could be in tree form or in any other form. But designing the data structure using composite pattern can easily manage the requirements for hierarchy structure, where any employee work as individual or may have a number of other employees as sub-ordinates.

4.3.2 When to Use

1. If we need to design a tree like structure where any node can be a leaf or a composite node again, and all this can be calculated dynamically at runtime.

2. When we want to make the system very generic, using a single interfaces which can accommodate primitive as well as composite elements.

4.3.3 Usage

1. It makes the system very generic.
2. It is easy to add a new component to any existing composite structure and the existing core framework can operate on new components as well without making any modification.
3. It is very easy to replace any existing component from the hierarchy.

4.3.4 Drawbacks

1. It results in over generic structure, which sometimes unable to enforce any validations rule on the components to be added.

4.4 Decorator Pattern

A decorator also known as wrapper is an object that has an interface identical to an object it contains. Any call that the decorator gets, it relays to the wrapped object and add its own functionalities along the way, either before or after the call. **OR**

The Decorator attaches additional responsibilities to an object dynamically. Although paintings can be hung on a wall with or without frames, frames are often added, and it is the frame which is actually hung on the wall. Prior to hanging, the paintings may be matted and framed, with the painting, matting, and frame forming a single visual component.

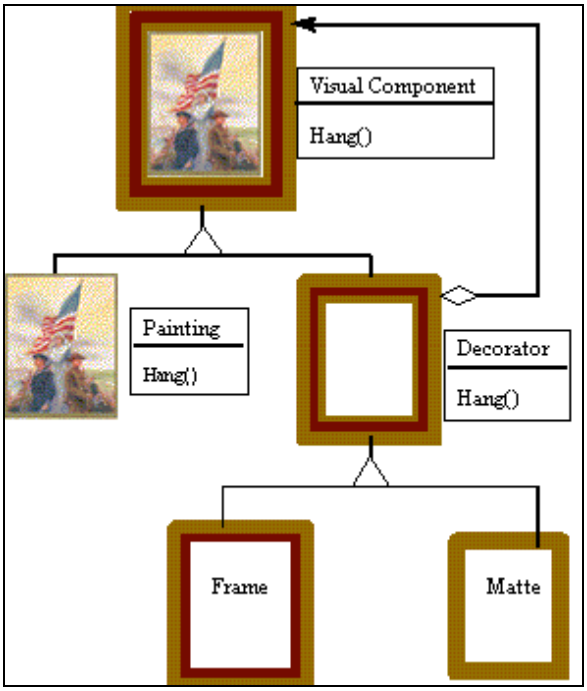

Figure 9: Object Diagram for Decorator using Framed Painting Example

4.4.1 Example

1. If we need to draw different borders for toolbar buttons, and one border may be applicable to more than one button. This can extend beyond border i.e. to other functionalities like color, font size etc.

2. Suppose if we are developing our data access system and want to track each and every database call made by using java.sql objects, we can define decorator connection, statement and result set which can wrap any other data base object and can intercept the calls made by these.

4.4.2 When to Use

1. When we need to add a common functionality to more than one class or objects.
2. When extending a class for adding some functionality may duplicate the efforts across the classes.

4.4.3 Usage

1. It is a very flexible way to add functionalities to any class without extending it.
2. The same functionality can be added to more than one class without duplicating the code and efforts.
3. It is applicable to both visual and non-visual programming elements.
4. It enables us to add functionalities to any existing class without extending it.

4.4.4 Drawbacks

1. Decorator and its enclosed objects are not identical so object type test can fail.
2. It results in a number of small objects in system which can be a maintenance headache for developer maintaining the system.

4.5 Façade Pattern

It is used to hide the complexities of system behind a simple interface.

Generally with time, the simple system grows to complex systems having a number of different APIs and objects to interact with. It happens especially if we are following the design patterns. Due to this increased complexities, it becomes very difficult for client to remember the exact flow of API and the right way to use it. Façade comes here and provides a simple interface to user, thus hides the complete complexity behind it.

OR

The Facade defines a unified, higher-level interface to a subsystem that makes it easier to use. Consumers encounter a Facade when ordering from a catalog. The consumer calls one number and speaks with a customer service representative. The customer service representative acts as a Facade, providing an interface to the order fulfillment department, the billing department, and the shipping department.

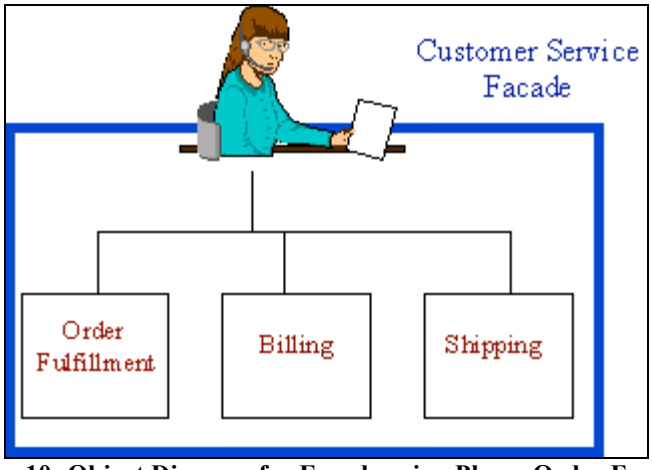

Figure 10: Object Diagram for Facade using Phone Order Example

4.5.1 Example

1. One way to access database is to use the raw API of JDBC, which may not be very convenient for every user, or user may need to repeat same code again and again to access the database. Other alternative is to create a 'Data Access Object', which can hide, the complexities of JDBC API and can provide a simple and convenient interfaced to user for most of the required operations. So we can define a SQL Data Access Manager, which can provide methods like execute query, get table names etc. Here SQL Data Access Manager is working as façade.
2. Any session bean on server most of the time works as façade.
3. An Object Manager on server, which allows the user to save any, object and fire the relevant events in a single call.

4.5.2 When to Use

1. When the underlying system is complex in nature and it needs more than one calls to complete any operation
2. When we want to hide the complexities of a system behind a simple interface so that bugs due to wrong flow of calls can be avoided.
3. When we want to provide convenient methods to users so that duplicate code can be avoided.

4.5.3 Usage

1. Helps to avoid the coding bugs due to wrong flow of calls for a complex system
2. Helps to avoid duplicate code, which may need to be written for implementing different functionalities using some common flow or system calls.
3. It does not prevent the advance user from going into deeper to more complex classes or flow.

4.6 Flyweight Pattern

Sometimes in system, a number of instances are required to represent the data even when all the instances are similar in structure and basic composition but with a few different parameters. Here using flyweight pattern, one instance can be used to represent the data by identifying the variable data and make the object configurable for this before every method call.

OR

The Flyweight uses sharing to support large numbers of objects efficiently. The public switched telephone network is an example of a Flyweight. There are several resources such as dial tone generators, ringing generators, and digit receivers that must be shared between all subscribers. A subscriber is unaware of how many resources are in the pool when he or she lifts the hand set to make a call. All that matters to subscribers is that dial tone is provided, digits are received, and the call is completed.

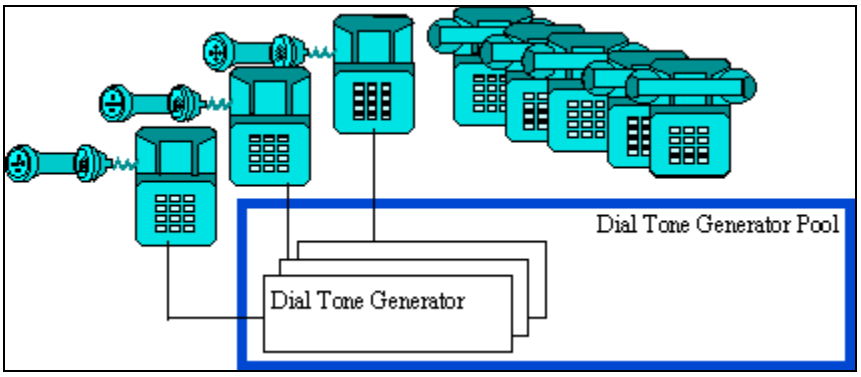

Figure 11: Dial Tone Generator Example of Flyweight

4.6.1 Example

1. In JTable, a column renderer serve the purpose to paint every cell on the table only table configures this renderer for its position, data and other variable attributes before using it for painting the next cell.
2. While painting a word on the screen, the same character instance is being used to paint complete word by just changing its location and data value.

4.6.2 When to Use

1. When large numbers of objects are required to represent the data and are similar in their composition and structure.
2. When basic structures of different instances are same and the variable parameters/data can be configured before operation.

4.6.3 Usage

1. Helps us to avoid the large number of object instances in memory

4.6.4 Drawbacks

1. Coding is not simple while implementing flyweight pattern for a complex system. Because some of the stale states can be carried forward by the shared instance and generally debugging becomes a headache for developer in these kinds of scenarios.

4.7 Proxy Pattern

It enables us to avoid the creation of resource consuming object until its actual need in the system. A complex and heavy object can be represented by a simple proxy object which creates the illusion of actual object but actually a proxy object of same type as of actual object. The proxy object initializes the actual object whenever it gets the first request from client. So it delays the initialization of heavy states until these are actually required.

OR

The Proxy provides a surrogate or place holder to provide access to an object. A check or bank draft is a proxy for funds in an account. A check can be used in place of cash for making purchases and ultimately controls access to cash in the issuer's account.

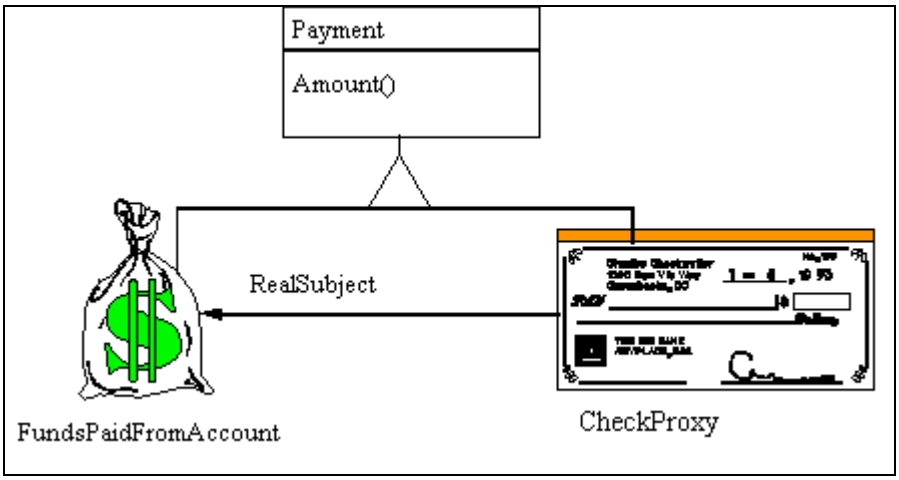

Figure 12: : Object Diagram for Proxy using Bank Draft Example

4.7.1 Example

1. In ORM tools, collections having a large number of items used to be initialized lazily using proxy pattern. Before actual initialization, only a proxy object creates the illusion of actual object.

4.7.2 Usage

1. Helps to avoid heavy object in memory before their actual requirement.
2. Helps to avoid heavy resource consuming calls like database access for large data until unless it is not required.

4.7.3 When to Use

1. When states of a heavy object can be initialized lazily, and are not bound with the instantiation of container object.
2. When the contained data needs a lot of memory and may also consume a lot resources for its initialization
3. When a resource consuming states needs to be initialized by some other system like database system or file system and are not saved as integral part of container object.
4. If the object is on remote machine and loading it takes a lot of time

4.7.4 Drawbacks

1. Special care need to be given if the container object is used as data transfer object across the tiers. In this scenario, the states of the proxy object must be initialized before detaching it from originating tier.

5 Behavioral Patterns

The Gang of Four has documented eleven behavioral patterns. Examples of these patterns can be found in coin sorting banks, restaurant orders, music, transportation, auto repair, vending machines, and home construction.

These patterns are more specifically concerned with communication between objects.

5.1 Chain of Responsibility

It allows a number of classes to attempt to handle a request, without any of them knowing about the existence or capabilities of other classes. So a lose coupling occurs between these classes and the request object passed to these classes is the only common link.

The request is passed through these classes until any one of these handles this request. Generally once handled, no other classes are visited. But a distorted form of this pattern allows more than one class to handle the request and process it, like servlets filters.

OR

The Chain of Responsibility pattern avoids coupling the sender of a request to the receiver, by giving more than one object a chance to handle the request. Mechanical coin sorting banks use the Chain of Responsibility. Rather than having a separate slot for each coin denomination coupled with receptacle for the denomination, a single slot is used. When the coin is dropped, the coin is routed to the appropriate receptacle by the mechanical mechanisms within the bank.

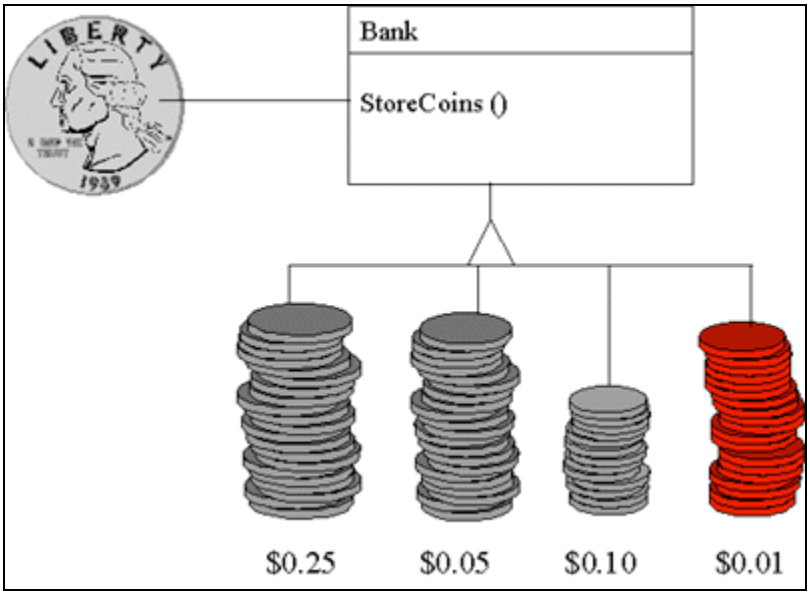

Figure 13: Object Diagram for Chain of Responsibility using Coin Sorting Example

5.1.1 Example

1. A help system, which can take the request for help from specific portion of UI. A chain of different help handlers in help system can try to handle this request and if any handler is successful to handle this request, it will do the processing. Further no handler will be invoked by core system.
2. We have the requirement to process some of the data from flat files and based on this data and some other parameters to the system, have to feed this data to database. The processing flow may vary depending on type of data and supplied parameters. Here distorted form of chain of responsibility can be applied.

5.1.2 Usage

1. It reduces the coupling among objects. Objects need not to be familiar with each other, they just should be able t o forward the request to next request handler.
2. It helps us in structured programming i.e. we can divide the responsibilities among the objects.
3. The request handlers can be added or removed dynamically based on some xml or flat configuration file.

5.1.3 When to Use

1. When processing logic for request may vary depending on request attributes or other parameters.
2. When more than one objects qualify to handle t he request.

3. When we want to add or remove the request handler objects dynamically i.e. want to make these highly configurable using configuration files.

4. When sequence of objects, handling the request, may need to change for different kind of requests.

5.1.4 Drawbacks

1. The request handlers must be implemented by an interface (especially if there are already extended from any other class). Here duplicate code will exists with the handlers to get the next handler and forward the request to it.

5.2 Command Pattern

The system chooses the right request handler for a request and forwards the request to it. Unlike chain of responsibility, where the request used to forward to multiple handlers and any qualified handler can handle it, here system finds the qualified request handler and hand over the request to it.

Point to consider is that all the request handlers i.e. command objects must be implemented by a common interface so that the system can choose and command without knowing its actual implementation and request it for processing the menu action.

OR

The Command pattern allows requests to be encapsulated as objects, thereby allowing clients to be parameterized with different requests. The "check" at a diner is an example of a Command pattern. The waiter or waitress takes an order, or command from a customer, and encapsulates that order by writing it on the check. The order is then queued for a short order cook. Note that the pad of "checks" used by different diners is not dependent on the menu, and therefore they can support commands to cook many different items.

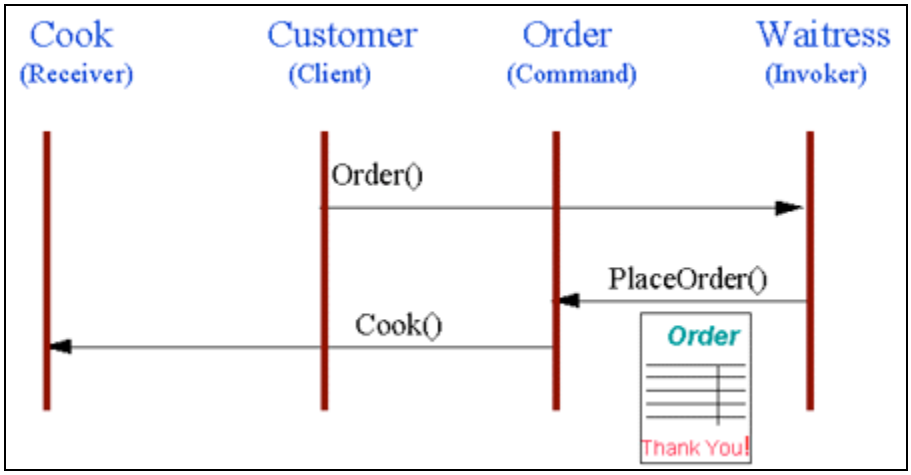

Figure 14: Object Interaction Diagram for Command using Diner Example

5.2.5 Example

1. A UI has a menu with different menu items, which can serve different purposes. One way to define t he action for menu items to define a request handler class and put the logic for all the items in this using if – else construct. But this could be a very crude, non-extensible and non-maintainable solution. Other solution is to define different request handlers for each menu item and choose the right one before asking for processing using Command Pattern.

2. If we need to implement Undo functionality, command pattern is the best strategy. A separate object can be created with all the information required to undo-redo the actions and can be put to the stack. Here objects are not known to each other and also not dependent on anyone, but are capable to do their own action based on the given command.

5.2.6 Usage

1. Helps us to define a structured system where every command object has code only for one action event.
2. New request handlers i.e. commands can be added easily to the system.
3. Client application is not dependent on the implementation of various actions, thus any change can be done easily.

5.2.7 When to Use

1. When we have a number of different requests.
2. When we want to abstract the implementation of different UI actions from UI.
3. When extensibility is the requirement.

5.3 Iterator Pattern

It is used to iterate over a collection of elements, without knowing what is contained in the list. It is the most simple pattern and used heavily.

A slight different implementation can have a filter with it, which can be used to filter out the desired elements from the collection.

OR

The Iterator provides ways to access elements of an aggregate object sequentially without exposing the underlying structure of the object. On early television sets, a dial was used to change channels. When channel surfing, the viewer was required to move the dial through each channel position, regardless of whether or not that channel had reception. On modern television sets, a next and previous button is used. When the viewer selects the "next" button, the next tuned channel will be displayed. Consider watching television in a hotel room in a strange city. When surfing through channels, the channel number is not important, but the programming is. If the programming on one channel is not of interest, the viewer can request the next channel, without knowing its number.

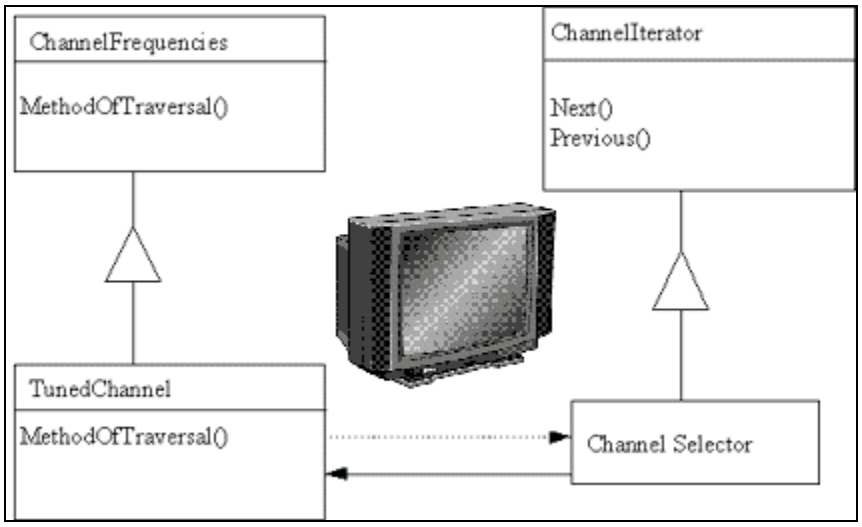

Figure 15: Object Diagram for Iterator using Channel Selector Example

5.3.1 Example

1. In java, we can ask the collection for iterator and can traverse the whole list using it.

5.3.2 Usage

1. Simple and convenient way to iterate over a collection of elements, or filtered elements depending on the requirement and implementation of iterator.

5.3.3 When to Use

1. When requirement is to iterate over a collection of elements

5.3.4 Drawbacks

1. Underlying data of iterator may get modified in a big system. In this case, any element may get skipped or read twice based on the implementation of storage data structure.

5.4 Mediator Pattern

In our systems, a number of classes exist to complete the functionality and with time, this number tends to increase. With increased number of classes, the major difficulty lies with communication among these classes. To communicate with each other, every class need to be aware about methods of any new/existing class which over the period of time makes the system very unstructured and tangled.

Mediator pattern provides the solution for this problem. It acts as mediator and now this is the only class which actually knows about the methods of all the classes. The classes, which need to communicate, have to interact with this mediator, which further passes on their message to concerned object.

OR

The Mediator defines an object that controls how a set of objects interacts. Loose coupling between colleague objects is achieved by having colleagues communicate with the Mediator, rather than with each other. The control tower at a controlled airport demonstrates this pattern very well. The pilots of the planes approaching or departing the terminal area communicate with the tower, rather than explicitly communicating with one another. The tower enforces the constraints on who can take off or land. It is important to note that the tower does not control the whole flight. It exists only to enforce constraints in the terminal area.

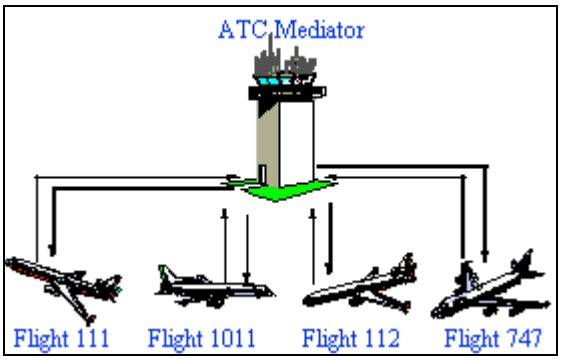

Figure 16: ATC Tower Example of Mediator

5.4.1 Example

1. We have a server side system with a workflow engine. This workflow engine used to execute different workflows based on the request from other components, like an Object Manager can save an object in database and request to this for execute some specific workflow engine. In the same manner, a UI action can request this engine to execute another workflow. This system is very extensible and tends to have a number of other system/application components, which need to be interacted with workflow engine. Here one way is to make every component familiar with API of workflow engine and also to make workflow engine familiar with every new component's API so that it can give the response. But it is very crude and may demand for code changes again and again. Best solution is to define a mediator, which can bind the component dynamically, and so workflow engine may listen or send the event to any component.

5.4.2 Usage

1. Mediator is the only class, which knows about other classes, so if any change is required in interface, the corresponding changes have to be done only in mediator.
2. Highly flexible system where n Number of new components can be added or removed from the system without actually affecting any API or code change.
3. It can produce a ripple effect in the system, even when no component is directly known to any other component. It can propagate the event produced by one component to n number of other components, which actually are not a ware about source component but registered for this event.
4. It makes loose coupling between the components.

5.4.3 When to Use

1. When we need to design a big extensible system where a number of new components are expected in future.

5.4.4 Drawbacks

1. Sometimes, it becomes very difficult to track that which even in the system changes the state and who was the originator of this event. So a proper auditing and logging system should be in place before using t his functionality in production.

5.5 Observer Pattern

It enables one or more objects to observe the states of one or more other objects. By implementing this pattern, one object request other object to intimate it for any change in a specific state which enables the system to refresh or act dynamically based on any state change anywhere in the system.

Here the object, which is producing the events for state change, is called observable and the object capturing the events is called observer.

OR

The Observer defines a one to many relationships, so that when one object changes state, the others are notified and updated automatically. Some auctions demonstrate this pattern. Each bidder possesses a numbered paddle that is used to indicate a bid. The auctioneer starts the bidding, and "observes" when a paddle is raised to accept the bid. The acceptance of the bid changes the bid price, which is broadcast to all of the bidders in the form of a new bid.

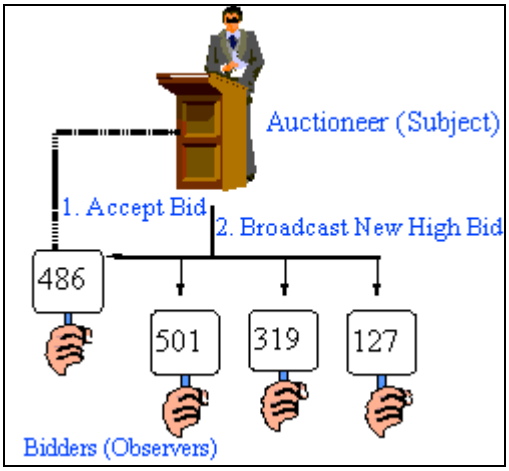

Figure 17: : Auction Example of Observer

5.5.1 Example

1. If an object contains the data for system and two different objects like JList and JTable are responsible to display this data in required format. Here JList and JTable can register themselves as observer to object containing the data. So that any change in data can be captured and corresponding change can be made in view.

5.5.2 Usage

1. It helps the objects to interact in a loosely fashion.
2. Any new observer can be added for any existing observable at any time without making any change in it.
3. The observable is completely abstract from the observers.

5.5.3 When to Use

1. When more than one object may be interested in the state of any object and this number may increase or decrease with time.

5.5.4 Drawbacks

1. A loss of even may occurs if observer is not active to observe the changes in observable or getting shut down due to any reason.
2. The system becomes very dynamic and sometimes debugging becomes a headache for developers in the absence of proper debugging strategies.

OR

It defines a way for classes to be loosely coupled and for one class (or many) to be notified when another is updated. Basically, this means that when something happens in one place, you notify anyone who is observing and that is interested in that one place.

There are **two ways** to look at the Observer pattern. The **first way** involves the Observer and Observable classes found in the **java.util** package. The **second way** follows the JavaBeans component model of registering event listeners with components.

One significant problem in using the classes (*Observer* and *Observable*) to implement the Observable pattern is that you have to extend Observable. This forces a class hierarchy structure that might not be possible in the single-inheritance world of the Java platform.

There are a number of complications you need to watch out for when using the Observer pattern. First is the possibility of a memory leak. The Subject maintains a reference to the Observer. Until the Subject releases the reference, the garbage collector cannot remove the Observer. Be aware of this possibility and remove observers where appropriate. Also note that, at least when registering event listeners the set of observer objects is maintained in an unordered collection. You don't necessarily know if the first registered listener is notified first or last. If you need to have cascading notifications, where objects A must be notified first, followed by object B, you must introduce an intermediary object to enforce the ordering. Simply registering the observers in a particular order will not enforce their order of notification.

Another area of the Java platform that models the Observer pattern is the Java Message Service (JMS), with its guaranteed delivery, non-local distribution, and persistence, to name a few of its benefits. The JMS publish-subscribe messaging model allows any number of subscribers to listen to topics of interest. When a message for the published topic is produced, all the associated subscribers are notified.

There are many other places in the Java platform that model the Observer pattern -- the pattern is frequently used throughout the Java platform.

5.6 Strategy Pattern

Strategy pattern consists of a number of related algorithms encapsulated in a driver class named as context. User has the flexibility to choose any of the algorithms and can ask the context to apply this algorithm to current process.

The various algorithms should implement a common interface so that these can be invoked transparently.

OR

A Strategy defines a set of algorithms that can be used interchangeably. Modes of transportation to an airport are an example of a Strategy. Several options exist, such as driving one's own car, taking a taxi, an airport shuttle, a city bus, or a limousine service. For some airports, subways and helicopters are also available as a mode of transportation to the airport. Any of these modes of transportation will get a traveler to the airport, and they can be used interchangeably. The traveler must chose the Strategy based on tradeoffs between cost, convenience, and time.

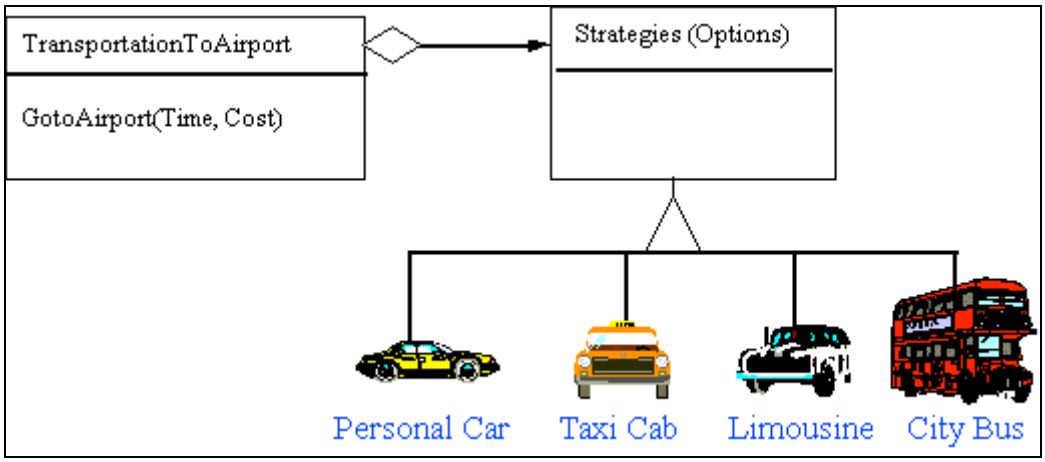

Figure 18: Object Diagram for Strategy using Airport Transportation Example

5.6.1 Example

1. If want to save files in different formats based on users choice.
2. If we are designing a UI and generally save it in database on server using some remote bean. In test environment, we may need to save it locally on file system. Here we can define a context with both persistence systems and switch between these two based on the scenario.

5.6.2 Usage

1. It allows us to select one of the several algorithms dynamically.

5.6.3 When to use

1. When we have more than options to perform an operation and want to switch over these options depending on runtime scenarios.

5.6.4 Drawbacks

1. Different algorithms may require different number of parameters for their operation. To support this, the strategy interface should be broad enough to pass parameters, which may not be required for one strategy but required for another.

5.7 Visitor Pattern

It provides a hook to one or more classes to act on the data of one class. It is against the OOP concepts but is required when we want to decouple commonly and repetitively used algorithms from a class to more generic classes. Here all or most of these classes, which are having some logic to act upon class data, can act on the class data.

What does ‘visiting’ mean? Generally an outside class can gain access to another class and that by calling its public methods. But here visiting each class means that you are calling a method already installed for this purpose, called accept. A visitor object is passed to this method, which if accepted by the class, its ‘visit’ method will be called passing itself as an argument.

OR

The Visitor pattern represents an operation to be performed on the elements of an object structure, without changing the classes on which it operates. This pattern can be observed in the operation of a taxi company. When a person calls a taxi company he or she becomes part of the company's list of customers. The company then dispatches a cab

to the customer (accepting a visitor). Upon entering the taxi, or Visitor, the customer is no longer in control of his or her own transportation, the taxi (driver) is.

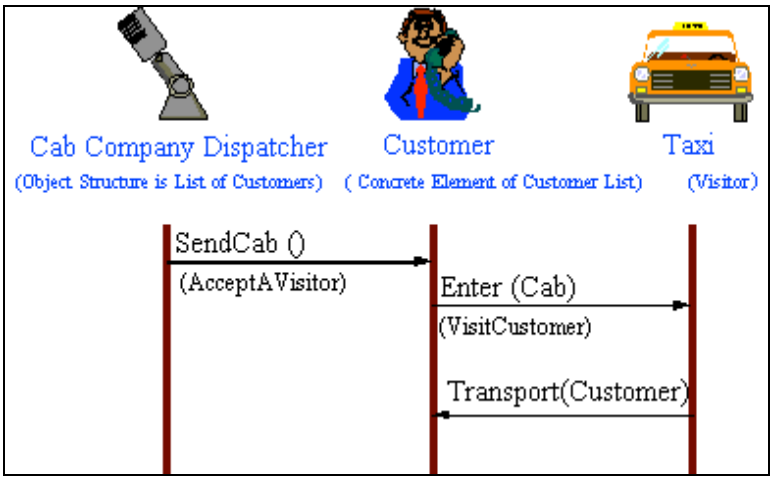

Figure 19: Object Interaction Diagram for Visitor using Taxi Cab Example

5.7.1 Example

1. We have a collection of employees, which comprises bosses as well. We want to calculate the total leaves taken by employees and bosses in last year. We can make two visitors one for only boss and other for employee not in a role of boss. Now simply iterate over collection and visit both visitors. Both visitors will calculate the respective leaves.

5.7.2 When to Use

1. When the program is in its mature stage and new classes are very unlikely.
2. When addition of new algorithms commonly required for a collection of classes
3. When we need to perform some operations on a composite structure

5.7.3 Usage

1. It adds functionalities to a collection of classes.
2. It is easy to add new operations to an existing program since only visitor code needs to be changed, which contains the logic.

5.7.4 Drawbacks

1. It can't act upon private methods so sometimes it might force to expose data through public methods.
2. Visitors are less helpful during the program's growth state because for every new class to which visitor needs to visit, a new methods have to be added to the visitor to match the signature with this new class.

5.8 Interpreter Pattern

The Interpreter pattern defines a grammatical representation for a language and an interpreter to interpret the grammar. Musicians are examples of Interpreters. The pitch of a sound and its duration can be represented in musical notation on a staff. This notation provides the language of music [14]. Musicians playing the music from the score are able to reproduce the original pitch and duration of each sound represented.

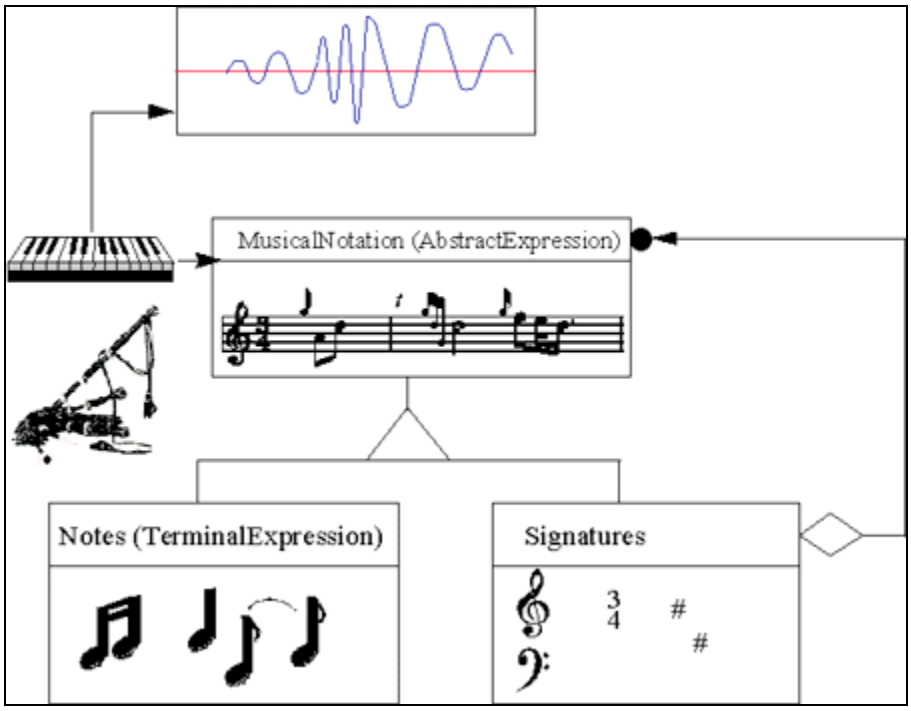

Figure 20: Object Diagram for Interpreter using Music Example

5.9 Memento Pattern

The Memento captures and externalizes an object's internal state, so the object can be restored to that state later. This pattern is common among do-it-yourself mechanics repairing drum brakes on their cars. The drums are removed from both sides, exposing both the right and left brakes. Only one side is disassembled, and the other side serves as a Memento of how the brake parts fit together [8]. Only after the job has been completed on one side is the other side disassembled. When the second side is disassembled, the first side acts as the Memento.

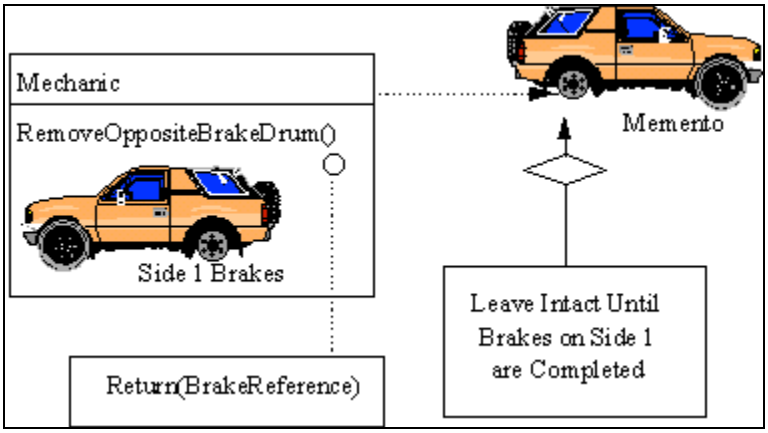

Figure 21: Object Diagram for Memento using Brake Example

5.10 State Pattern

The State pattern allows an object to change its behavior when its internal state changes. This pattern can be observed in a vending machine. Vending machines have states based on the inventory, amount of currency deposited, the ability to make change, the item selected, etc. When currency is deposited and a selection is made, a vending machine will deliver a product and no change, deliver a product and change, deliver no product due to insufficient currency on deposit, or deliver no product due to inventory depletion.

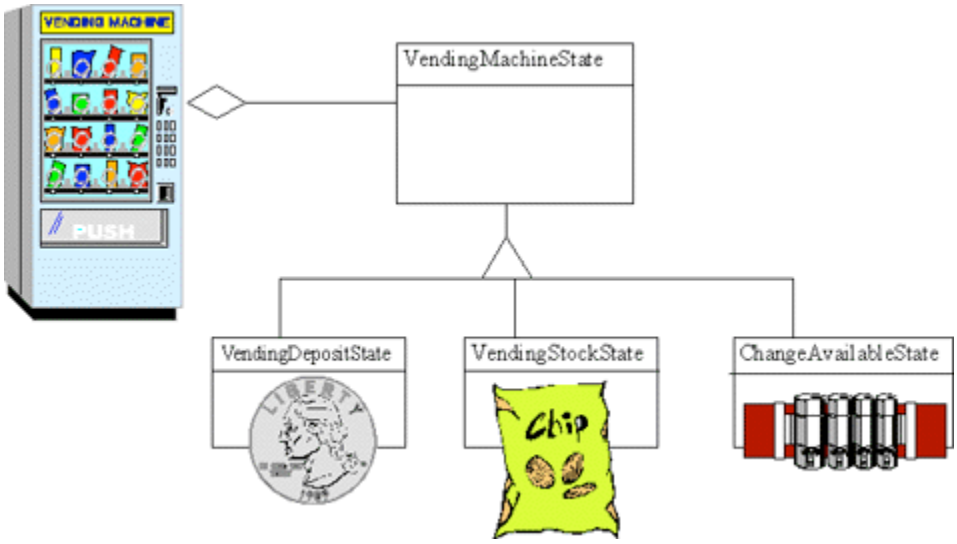

Figure 22: Object Diagram for State using Vending Machine Example

5.11 Template Pattern

The Template Method defines a skeleton of an algorithm in an operation, and defers some steps to subclasses. Homebuilders use the Template Method when developing a new subdivision. A typical subdivision consists of a limited number of floor plans, with different variations available for each floor plan. Within a floor plan, the foundation, framing, plumbing, and wiring will be identical for each house. Variation is introduced in the latter stages of construction to produce a wider variety of models.

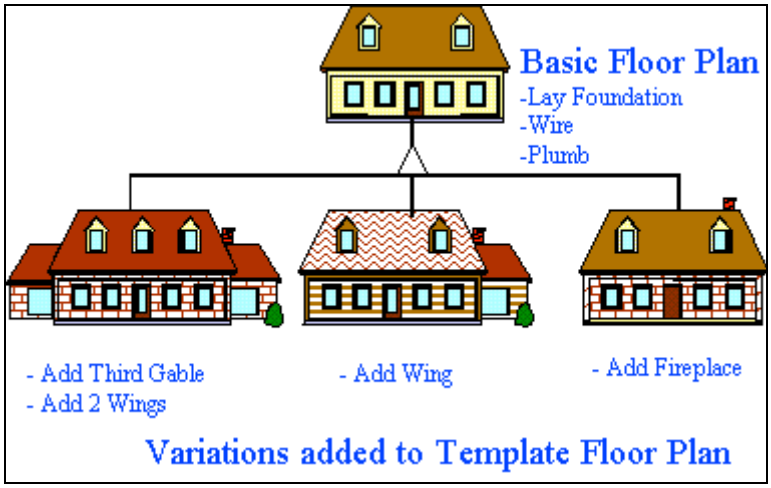

Figure 23: Basic Floor Plan Example of Template Method

D:\Data\Rohtash\
Tutorials\Java\Design